Importing Libraries

In [89]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report , confusion_matrix
from sklearn.model_selection import GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
```

Uploading the dataset

In [90]:
```
data = pd.read_csv(r"C:\Users\kadam\OneDrive\Desktop\Msc IT Fintech sem 4 (Project )\Elevate lab\ecommerce_returns_sy
```

In [92]:
```
data.head()
```

Out[92]:

	Order_ID	Product_ID	User_ID	Order_Date	Return_Date	Product_Category	Product_Price	Order_Quantity	Re
0	ORD00000000	PROD00000000	USER00000000	2023-08-05	2024-08-26	Clothing	411.59	3	Cl
1	ORD00000001	PROD00000001	USER00000001	2023-10-09	2023-11-09	Books	288.88	3	
2	ORD00000002	PROD00000002	USER00000002	2023-05-06	NaN	Toys	390.03	5	
3	ORD00000003	PROD00000003	USER00000003	2024-08-29	NaN	Toys	401.09	3	
4	ORD00000004	PROD00000004	USER00000004	2023-01-16	NaN	Books	110.09	4	

In [94]:
```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 17 columns):
 #   Column            Non-Null Count  Dtype  
---  ------            --------------  -----  
 0   Order_ID          10000 non-null  object 
 1   Product_ID        10000 non-null  object 
 2   User_ID           10000 non-null  object 
 3   Order_Date        10000 non-null  object 
 4   Return_Date       5052 non-null   object 
 5   Product_Category  10000 non-null  object 
 6   Product_Price     10000 non-null  float64
 7   Order_Quantity    10000 non-null  int64  
 8   Return_Reason     5052 non-null   object 
 9   Return_Status     10000 non-null  object 
 10  Days_to_Return    5052 non-null   float64
 11  User_Age          10000 non-null  int64  
 12  User_Gender       10000 non-null  object 
 13  User_Location     10000 non-null  object 
 14  Payment_Method    10000 non-null  object 
 15  Shipping_Method   10000 non-null  object 
 16  Discount_Applied  10000 non-null  float64
dtypes: float64(3), int64(2), object(12)
memory usage: 1.3+ MB
```

In [97]:
```
data.describe()
```

Out[97]:

	Product_Price	Order_Quantity	Days_to_Return	User_Age	Discount_Applied
count	10000.000000	10000.000000	5052.000000	10000.000000	10000.000000
mean	252.369307	3.006100	1.453682	44.195000	24.992162
std	142.883865	1.406791	297.983208	15.311983	14.363396
min	5.010000	1.000000	-719.000000	18.000000	0.000000
25%	128.650000	2.000000	-214.000000	31.000000	12.752500
50%	250.445000	3.000000	1.000000	44.000000	24.840000
75%	377.837500	4.000000	218.000000	57.000000	37.605000
max	499.890000	5.000000	726.000000	70.000000	50.000000

In [99]:
```
data.shape
```

Out[99]: (10000, 17)

Finding the null values

In [101…
```
data.isnull().sum()
```

Out[101…

```
Order_ID               0
Product_ID             0
User_ID                0
Order_Date             0
Return_Date         4948
Product_Category       0
Product_Price          0
Order_Quantity         0
Return_Reason       4948
Return_Status          0
Days_to_Return      4948
User_Age               0
User_Gender            0
User_Location          0
Payment_Method         0
Shipping_Method        0
Discount_Applied       0
dtype: int64
```

In [103…

```
data['Return_Flag'] = data['Return_Date'].notnull().astype(int)
```

In [105…

```
data['Return_Reason'] = data['Return_Reason'].fillna('No Return')
data['Days_to_Return'] = data['Days_to_Return'].fillna(0)
```

In [107…

```
data.isnull().sum()
```

Out[107…

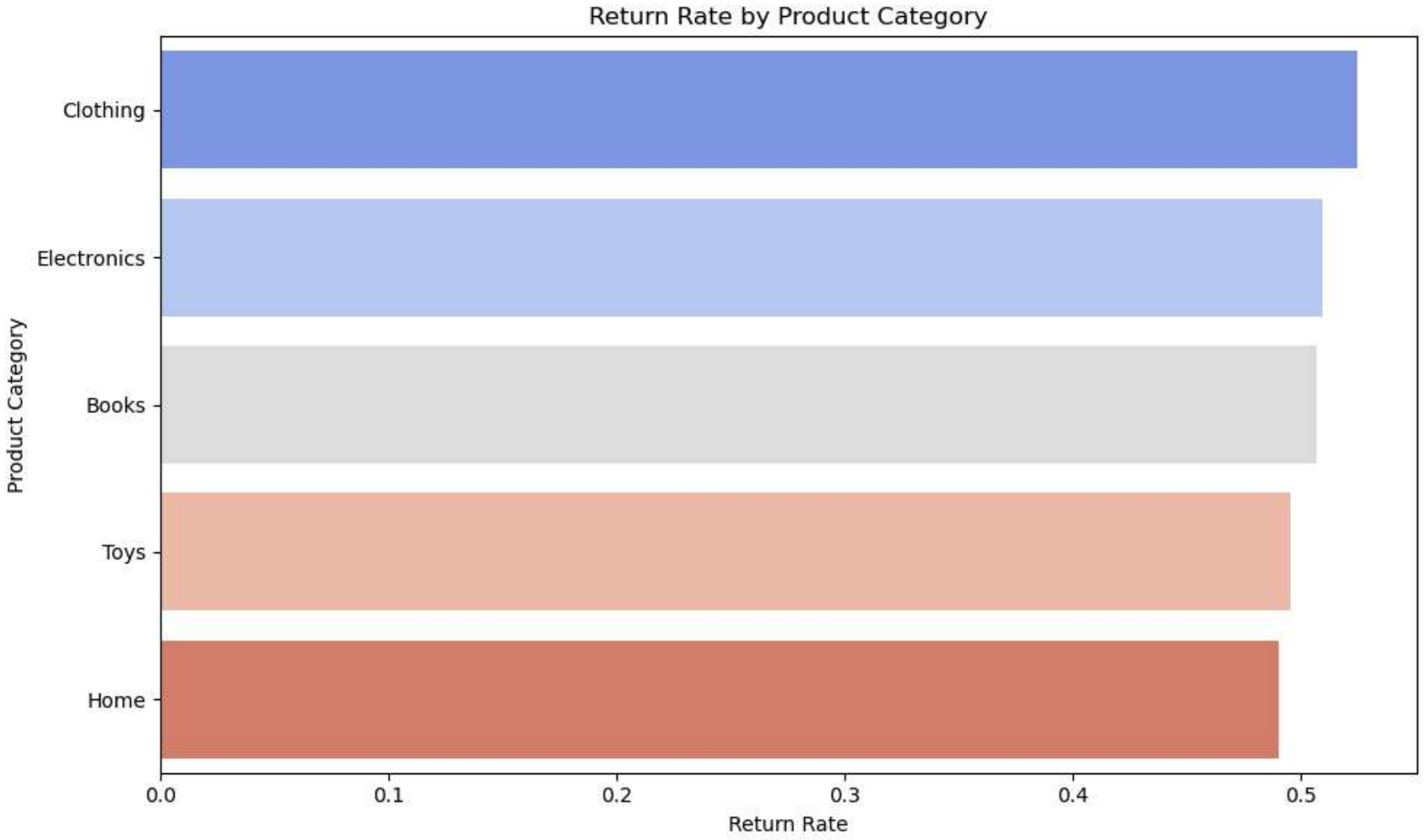
```
Order_ID               0
Product_ID             0
User_ID                0
Order_Date             0
Return_Date         4948
Product_Category       0
Product_Price          0
Order_Quantity         0
Return_Reason          0
Return_Status          0
Days_to_Return         0
User_Age               0
User_Gender            0
User_Location          0
Payment_Method         0
Shipping_Method        0
Discount_Applied       0
Return_Flag            0
dtype: int64
```

EDA

In [109…

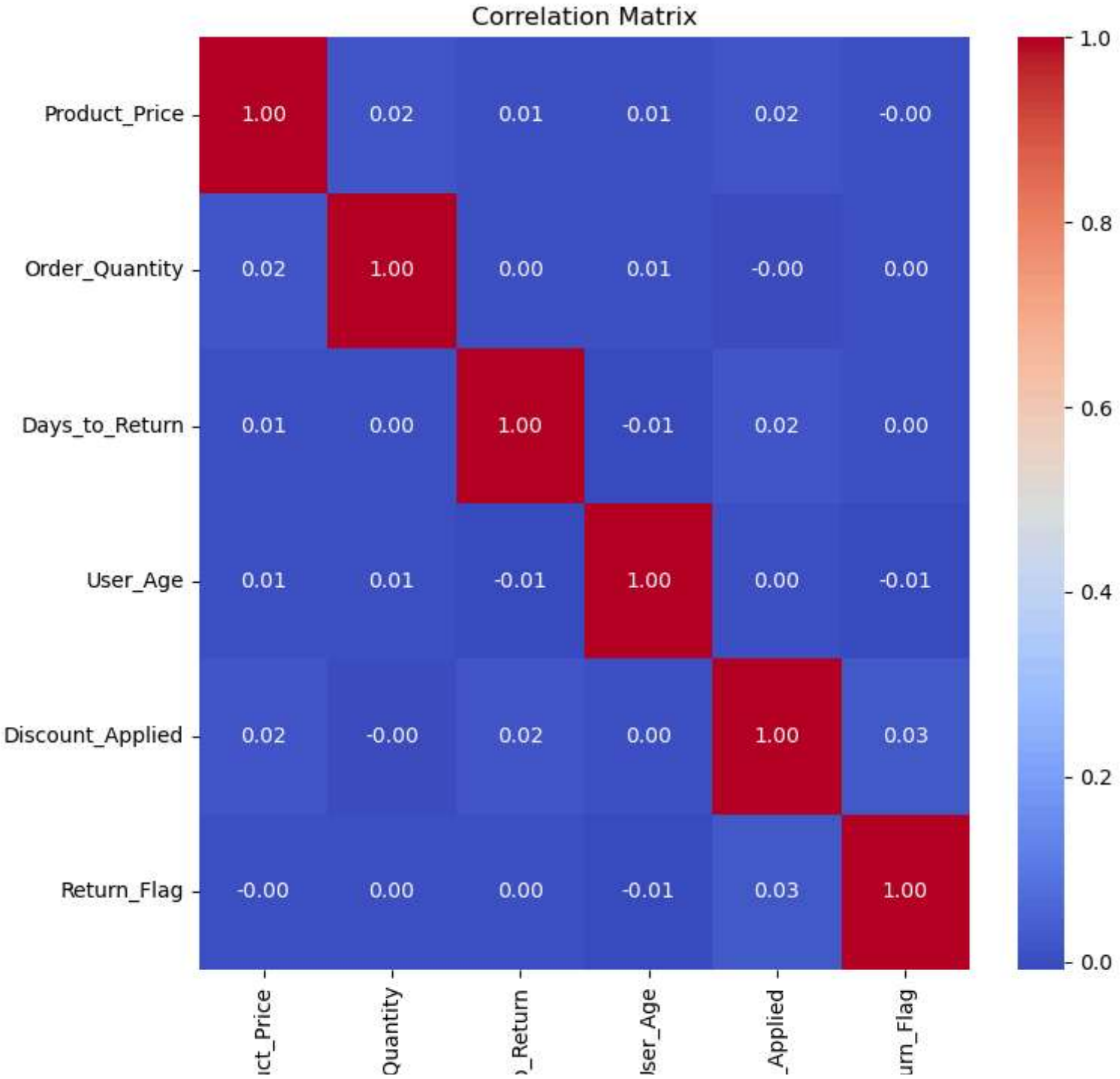
```
category_returns = data.groupby('Product_Category')['Return_Flag'].mean().sort_values(ascending=False)

# Plot
plt.figure(figsize=(10, 6))
sns.barplot(x=category_returns.values, y=category_returns.index, palette='coolwarm')
plt.title("Return Rate by Product Category")
plt.xlabel("Return Rate")
plt.ylabel("Product Category")
plt.tight_layout()
plt.show()
```

```
C:\Users\kadam\AppData\Local\Temp\ipykernel_21328\4086688495.py:6: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `h
ue` and set `legend=False` for the same effect.

  sns.barplot(x=category_returns.values, y=category_returns.index, palette='coolwarm')
```



In [110…

```
plt.figure(figsize=(8, 8))
sns.heatmap(data.corr(numeric_only=True), annot=True, cmap='coolwarm', fmt=".2f")
plt.title("Correlation Matrix")
plt.tight_layout()
plt.show()
```

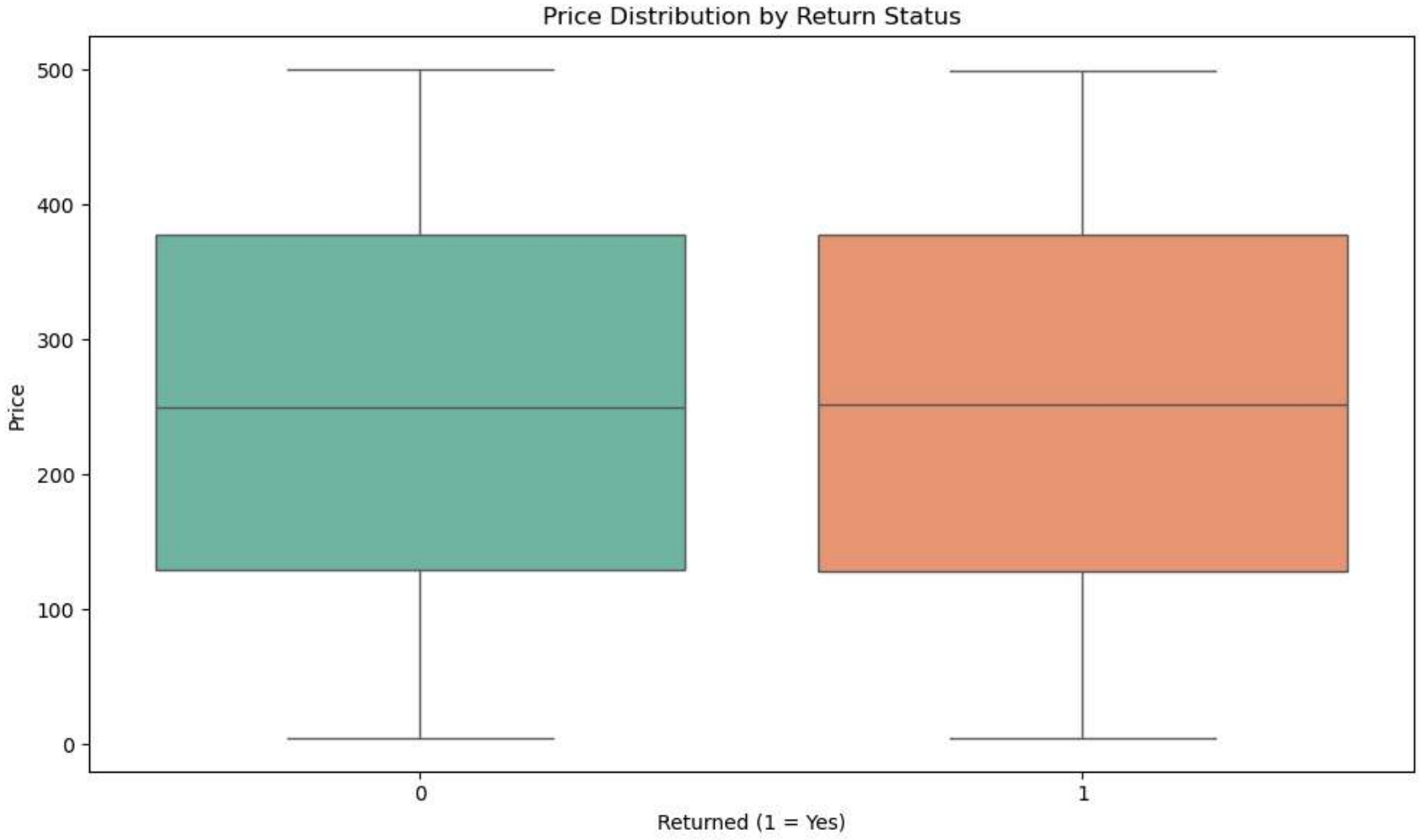

Correlation Matrix
Product_Price
Order_Quantity
Days_to_Return
User_Age
Discount_Applied
Return_Flag
1.00 0.02 0.01 0.01 0.02 -0.00
0.02 1.00 0.00 0.01 -0.00 0.00
0.01 0.00 1.00 -0.01 0.02 0.00
0.01 0.01 -0.01 1.00 0.00 -0.01
0.02 -0.00 0.02 0.00 1.00 0.03
-0.00 0.00 0.00 -0.01 0.03 1.00
1.0
0.8
0.6
0.4
0.2
0.0

Produ Order_ Days_tc U Discount_ Reti

In [111…

```
plt.figure(figsize=(10, 6))
sns.boxplot(data=data, x='Return_Flag', y='Product_Price', palette='Set2')
plt.title("Price Distribution by Return Status")
plt.xlabel("Returned (1 = Yes)")
plt.ylabel("Price")
plt.tight_layout()
plt.show()
```

```
C:\Users\kadam\AppData\Local\Temp\ipykernel_21328\2499472131.py:2: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `h
ue` and set `legend=False` for the same effect.

  sns.boxplot(data=data, x='Return_Flag', y='Product_Price', palette='Set2')
```



In [113…

```
data.head()
```

Out[113...

	Order_ID	Product_ID	User_ID	Order_Date	Return_Date	Product_Category	Product_Price	Order_Quantity	Re
0	ORD00000000	PROD00000000	USER00000000	2023-08-05	2024-08-26	Clothing	411.59	3	Cl
1	ORD00000001	PROD00000001	USER00000001	2023-10-09	2023-11-09	Books	288.88	3	
2	ORD00000002	PROD00000002	USER00000002	2023-05-06	NaN	Toys	390.03	5	
3	ORD00000003	PROD00000003	USER00000003	2024-08-29	NaN	Toys	401.09	3	
4	ORD00000004	PROD00000004	USER00000004	2023-01-16	NaN	Books	110.09	4	

In [121...
```
data.columns
```

Out[121...
```
Index(['Order_ID', 'Product_ID', 'User_ID', 'Order_Date', 'Return_Date',
       'Product_Category', 'Product_Price', 'Order_Quantity', 'Return_Reason',
       'Return_Status', 'Days_to_Return', 'User_Age', 'User_Gender',
       'User_Location', 'Payment_Method', 'Shipping_Method',
       'Discount_Applied', 'Return_Flag'],
      dtype='object')
```

Featurs Selecting

In [119...
```
df = data.drop(['Order_ID','Product_ID' , 'User_ID' ,  'Order_Date'  ,'Return_Date' , 'Return_Reason' ,
       'Return_Status' , 'Days_to_Return'] , axis = 1)
```

Encoding the character

In [75]:
```
cat_cols = df.select_dtypes(include=['object']).columns

label_encoders = {}
for col in cat_cols:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col].astype(str))
    label_encoders[col] = le
```

In [77]:
```
df.head()
```

Out[77]:

	Product_Category	Product_Price	Order_Quantity	User_Age	User_Gender	User_Location	Payment_Method	Shipping_Method
0	1	411.59	3	58	1	50	1	1
1	0	288.88	3	68	0	84	0	0
2	4	390.03	5	22	0	24	1	1
3	4	401.09	3	40	1	95	3	1
4	0	110.09	4	34	0	79	2	2

Split into X and Y

In [79]:
```
X = df.drop('Return_Flag', axis=1)
y = df['Return_Flag']

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42, stratify=y)
```

In [81]:
```
scaler = StandardScaler()
scaler.fit(X_train)
X_train = scaler.transform(X_train)
X_test = scaler.transform(X_test)
```

In [83]:
```
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
```

Out[83]:

LogisticRegression

```
LogisticRegression(max_iter=1000)
```

In [85]:
```
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred))
```

```
Accuracy: 0.5043333333333333
Confusion Matrix:
 [[560 924]
 [563 953]]
Classification Report:
              precision    recall  f1-score   support

           0       0.50      0.38      0.43      1484
           1       0.51      0.63      0.56      1516

    accuracy                           0.50      3000
   macro avg       0.50      0.50      0.50      3000
weighted avg       0.50      0.50      0.50      3000
```

In [87]:

```python
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('logreg', LogisticRegression(solver='liblinear'))
])

param_grid = {
    'logreg__penalty': ['l1', 'l2'],
    'logreg__C': [0.01, 0.1, 1, 10],
    'logreg__max_iter': [500, 1000]
}


grid = GridSearchCV(pipeline, param_grid, cv=5, scoring='accuracy', verbose=1, n_jobs=-1)
grid.fit(X_train, y_train)

print("Best Parameters:", grid.best_params_)
print("Best CV Score:", grid.best_score_)
print("Test Accuracy:", grid.best_estimator_.score(X_test, y_test))
```

```
Fitting 5 folds for each of 16 candidates, totalling 80 fits
Best Parameters: {'logreg__C': 1, 'logreg__max_iter': 500, 'logreg__penalty': 'l1'}
Best CV Score: 0.5014285714285714
Test Accuracy: 0.5033333333333333
```

In [ ]: